Some Pointers

Be specific

Don’t describe generic agents.
Describe what your agent does!

- **Sequence Item:** Contains the generated data from the Sequencer
- **Monitor:** Observes the inputs and outputs of the components.
- **Scoreboard:** Keeps track of results by comparing outputs to expected values generated from a reference model.

- **Driver:** Generates APB read/write stimulus, controlling address, data, and mode selection (Input, Kernel, Read). For the Convolution Accelerator, it triggers control signals, and manages RAMC handshake initiation.

- **Sequencer:** Coordinates between random and directed sequences, generating APB command streams and convolution control sequences. Ensures the randomization of kernel data, IFM inputs, and operation timing.

Be specific

Coverage Points

- All important `apb_wrapper` signals
- All important `convolver` signals

The **motivation** in probably more important than the signal itself. The signals are privy to change, the motivations not so much.

Coverage Points

For APB-SRAM

- **Reset Transitions Coverage:** Cover both reset assertion and deassertion across all FSM states to ensure proper initialization and recovery.
- **Data Coverage:** Transaction data values are sampled across ranges: low, mid, high, all-zeros, all-ones. Ensures coverage of typical and edge-case data patterns.
- **Address Coverage:** Access each region and include boundary addresses to validate address decoding

Motivate choices

A good test description should motivates **why** it is performed.

Test C1 — Impulse image (delta)

Purpose: kernel loading + identity placement
Setup: Input X is a 2D delta (single 1 at (r,c), zeros elsewhere). Kernel H arbitrary.
Expected: Output Y should equal H positioned starting at (r,c) in the full-convolution output (or centered if “same” mode).
Pass: Y matches H (or expected placement) within numeric tolerance.

Stay organized

Execute Stage

Test ID	Test Name	Description	Expectation	Module	Method	Inputs	Outputs	Status	Comments	Targeted Coverage	Date completed	Owner
									are randomized.	[ALU1]		
ALU_01	ALU ARITMETIC EDGE CASES	Test all aritmetic operations with maximal and minimal values for the inputs	All results should be computed corrrectly	alu.sv	FPV & Constrained Randor	TBD	TBD	Untested		All valid operands crossed with edge cases for the operand and output values. [ALU2]		
REG_BLOCK_00	REG BLOCK GENERAL	Test all combinations of valid control signals with randomized data	All outputed data should be correct	reg_block.sv	Constrained Random	TBD	TBD	Untested		All legal control input combinations crossed. Evenly spaced numberic ranges sould be covered for all outputs. [REG1-5]		
REG_BLOCK_01	REG BLOCK DATAFLOW	Testing that dataflow into/between/out of registers are correct	dataflow should be as specified	reg_block.sv	FPV	TBD	TBD	Untested	Translate specifications to datatransfer SVA.	Data transfer [REG1-6]		
REG_BLOCK_02	REG BLOCK RESET	Test reset during operation	All outputs should be 0 after reset	reg_block.sv	FPV & Constrained Randor	TBD	TBD	Untested	First *OP_code* and data are randomized to simulate operation. After reset *shift* and *mem_write* should be used for accessing all registers after reseting without writing new data.	Sufficient number of *shift* and *mem_write* crossed with reading zeros [REG6]		

DoD – Definition of Done

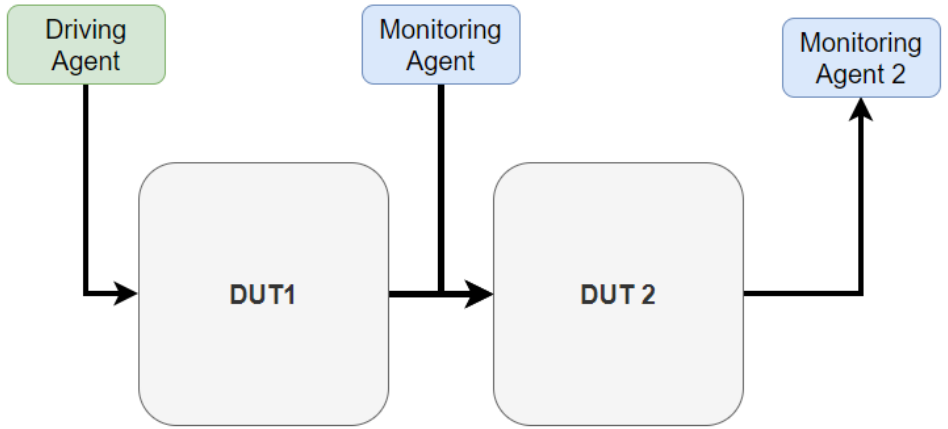
Make sure your definition of done is specified, quantifiable and completable **reasonably**

Verification Closure Criteria

- All functional coverage points reached ($\geq 95\%$).
- Code coverage $\geq 90\%$.
- All assertions pass without failure.
- No mismatches reported by scoreboard.
- All directed corner cases verified.

Draw!

Even a crude image made in draw.io can impart a lot of knowledge on how a test environment is structured



Some other points

- Consider this a living document. You may change it…
- Reference the design specification – these two are sibling documents.
- If it doesn’t feel like it’s helping you, it’s probably not complete.